Assignment 3: Inverse Differential Kinematics

Robot Kinematics and Dynamics

Prof. Jeff Ichnowski
Albert Xu
Henry Liao
Karthik Srinivasan

Due: See Canvas for the due date

Contents

1 Overview

This assignment reinforces the following topics:

- Jacobians
- Psuedoinverses

2 Background

2.1 The Inverse Problem

So far we have studied the functions that perform forward kinematics, which means going from a manipulator's configuration space, Q to its workspace, W:

$$W = f(Q)$$

In other words, we have determined the function for a robot arm that uses its joint angles to calculate the position and orientation of its end effector (as well as other frames on the robot).

You can probably think of a lot of cases where we want to do the inverse: given a specified effector position (and potentially orientation), what are the joint angles that will achieve this? This inverse process is problematic because the forward kinematics function, f, is (usually) non-linear. That means that we (usually) cannot just say:

$$Q = f^{-1}(W)$$

This means that finding the inverse function for the kinematics of an arm is difficult (but by no means impossible—in the next assignment we will tackle several methods to do this).

2.2 Differential IK

While the general problem of going from workspace positions to joint angles is difficult, we can begin with the *differential* problem – going from workspace velocities to configuration space velocities.

$$\dot{q} = f^{-1}(\dot{w})$$

From the last assignment, you know that the function relating these two sets of velocities is the Jacobian, J. A key property of the Jacobian is that it is linear (given the configuration q of the arm). In practical terms, this means that if we know an arm's joint angles and desired instantaneous end-effector velocities, we can calculate the required joint velocities by inverting the Jacobian:

$$\dot{q} = J^{-1}(q)\dot{w}$$

Because (for a given configuration) the Jacobian can be represented as a matrix, this is a much easier problem than the general IK problem we began with. It doesn't directly give us the joint angles for a given end effector position, but it can be used to help solve for these, as we'll see in the assignment.

2.3 Pseudoinverse

But wait—although inverting matrices is easy for computers, not all matrices (and not all Jacobians) are invertible! In order for a matrix to be invertible, it has to be square, and have a non-zero determinant. In terms of a robot arm this means that it has to be:

- Fully constrained (the same number of workspace and configuration space variables)

- Non-singular (the arm's instantaneous motion can span the dimension of the workspace, meaning J is full rank)

Fortunately, mathematicians have developed a technique called the *pseudoinverse* to help get around the problem of not having a square or full rank matrix. The pseudoinverse has some, but not all, of the properties of a true matrix inverse. The general method to compute this quantity involves a technique called *singular value decomposition*, but when the matrix is *full rank*, there are analytical formulas that can be used to find the inverse.

In this case, we basically make the matrix square by multiplying it by its transpose. We then invert the new matrix, and multiply the result by the transpose of the original matrix to get the dimensions to match up. The exact order of multiplication depends on the dimension of the matrix:

2.3.1 Underconstrained

To determine the joint velocities that result in a given a workspace velocity, we need to solve a system of linear equations (given a joint configuration). If there are n joint variables and m workspace variables, then the Jacobian has dimensions $m \times n$. Consider the case where $m < n$ i.e. there are more joints than the number of workspace variables. This results in a system of m equations (i.e. constraints) in n unknowns (i.e. joint velocities) where $m < n$, hence the name underconstrained.

Assuming the Jacobian is full rank, such a system always has a solution. This means we can always find joint velocities that result in a given end effector velocity. However, there is a whole family of solutions that satisfy these constraints. Therefore, one might be interested in searching the best solution in this family, measured with respect to some criterion. For the purpose of this discussion, we are interested in solutions (joint velocities) which result in the smallest effort in moving the arm. We can quantify this effort using the squared norm of the joint velocities.

Posing this as an optimization problem, we want to find $\dot{q}$ such that we minimize $\dot{q}^T \dot{q}$ satisfying $\dot{w} = J(q)\dot{q}$, where $\dot{w}$ and $J(q)$ are known.

This optimization problem is equivalent to a least-norm problem for linear systems of equations and has an exact analytical solution given by:

$$\dot{q} = J^{+}\dot{w},$$

where J^{+} is known as the right pseudoinverse of the Jacobian and is given by:

$$J^{+} = J^T (JJ^T)^{-1}$$

2.3.2 Overconstrained

This case is when the number of rows the Jacobian has is greater than (or equal to) the number of columns. In other words, there are more equations than unknowns.

In this case, the arm cannot move the end effector with the exact desired velocity, but this solution will try to solve for the "best" joint velocities in terms of the sum-squared error of end effector velocities. This is called the left puesdoinverse.

$$J^{\#} = (J^T J)^{-1} J^T$$

As a side note, the pseudoinverse in this overconstrained case is really useful even outside of kinematics, and is used to quickly and reliably do things like solve for best-fit lines to data.

2.3.3 Perfectly Constrained

Note that in the case of a square full rank matrix, the psuedoinverse will result in J^{-1}.

2.4 Course Logistics

In this class you would be using the following websites regularly:

- Canvas: The main course web page. This links to everything relevant to the class (including the next two links).
- Piazza: This is the best way to ask course staff questions, and see course announcements.
- Gradescope: This is where you will be turning in your homework. You should have already been added to the course. If you haven't, please contact the course staff via email or Piazza post.

You can write your `writeup.pdf` in Overleaf: upload the assignment zip as a new project, and use `assignment3.tex` as your template (it pulls in `16384_doc.cls` and the figures from the `latex/` folder automatically).

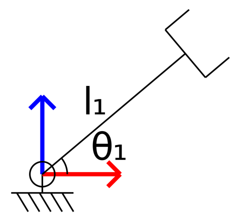
3 Instructions

- See Canvas for the due date.
- There are two separate submissions to Gradescope: your written `writeup.pdf` and your code, zipped as `<andrew_id>_hw3.zip`. Run `create_submission.py` to generate the code zip. See the complete submission checklist at the end, to ensure you have everything.
- You are free to write `writeup.pdf` using any method, but we recommend LaTeX for consistent formatting. You can upload the assignment zip to Overleaf and edit `assignment3.tex` directly.
- Each question (for points) is marked with a **points** heading.
- **Start early!** This homework may take a long time to complete.
- **During submission indicate the answer/page correspondence carefully when submitting on Gradescope.** If you skip a written question, just submit a blank page for it. This makes our work much easier to grade.
- If you have any questions or need clarifications, please post in Piazza or visit the TAs during the office hours.
- Unless otherwise specified, **all units are in radians, meters, and seconds**, where appropriate.

4 Written Questions

1) R Robot

Consider the following robot



Assume $\theta_1 = \frac{\pi}{4}$.

a. [1 point] What is the dimension of the end-effector Jacobian? Consider end-effector positions $\begin{bmatrix} x \\ y \end{bmatrix}$ in $\mathbb{R}^2$.

b. [1 point] Is this system underconstrained, overconstrained, or neither? Why?

c. [2 points] Draw the velocity vector for the end effector caused by joint 1 moving with unit velocity. The direction of these vectors should be correct, but do not worry about the magnitude. You can complete this problem by inspection or by using properties of the Jacobian.

d. [2 points] What is the dimension of the space spanned by this vector?

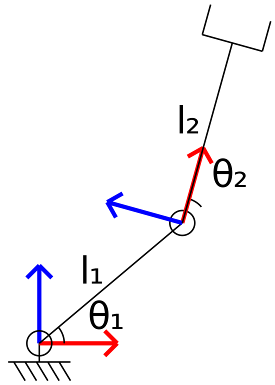
e. [1 point] What is the rank of J for this configuration?

f. [1 point] When identifying the inverse differential kinematics for this problem, would you use the inverse, the left psuedoinverse, or the right psuedoinverse?

g. [3 points] What are the benefits of a left pseudoinverse versus a right pseudoinverse?

2) RR Robot

Consider the following robot



Assume $\begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} = \begin{bmatrix} \frac{\pi}{4} \\ \frac{\pi}{4} \end{bmatrix}$.

a. [1 point] What is the dimension of the end-effector Jacobian? Consider end effector positions $\begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$.

b. [1 point] Is this system underconstrained, overconstrained, or neither? Why?

c. [4 points] On separate plots, draw the velocity vector for the end effector caused by:

(i) Joint 1 moving with unit velocity (joints 2 has zero velocity).

(ii) Joint 2 moving with unit velocity (joints 1 has zero velocity).

The direction of these vectors should be correct, and the relative magnitudes should be approximately correct, but do not worry about exact magnitudes. You can complete this problem by inspection or by using properties of the Jacobian.

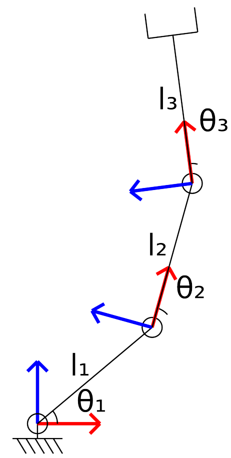
d. [2 points] What is the dimension of the space spanned by these vectors? Are the 1st and 2nd vectors linearly independent?

e. [1 point] What is the rank of J for this configuration?

f. [1 point] When identifying the inverse differential kinematics for this problem, would you use the inverse, the left psuedoinverse, or the right psuedoinverse?

3) RRR Robot

Consider the following robot



Assume $\begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_2 \end{bmatrix} = \begin{bmatrix} \frac{\pi}{3} \\ \frac{\pi}{3} \\ \frac{\pi}{3} \end{bmatrix}$.

a. [1 point] What is the dimension of the end-effector Jacobian? Consider end effector positions $\begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$.

b. [1 point] Is this system underconstrained, overconstrained, or neither? Why?

c. [6 points] On separate plots, draw the velocity vector for the end effector caused by
   (i) Joint 1 moving with unit velocity (joints 2 and 3 have zero velocity).
   (ii) Joint 2 moving with unit velocity (joints 1 and 3 have zero velocity).
   (iii) Joint 3 moving with unit velocity (joints 1 and 2 have zero velocity).

   The direction of these vectors should be correct, and the relative magnitudes should be approximately correct, but do not worry about exact magnitudes. You can complete this problem by inspection or by using properties of the Jacobian.

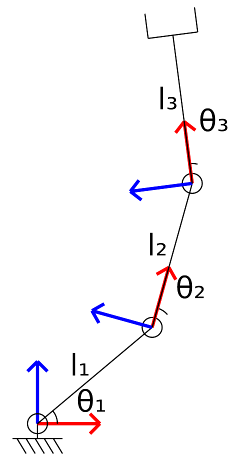
d. [2 points] What is the dimension of the space spanned by these vectors? Are the 1st and 2nd vectors linearly independent? The 1st and 3rd? The 2nd and 3rd? All three?

e. [1 point] What is the rank of J for this configuration?

f. [1 point] When identifying the inverse differential kinematics for this problem, would you use the inverse, the left psuedoinverse, or the right psuedoinverse?

4) RRR Robot, revisited with rotation

Consider the following robot



a. [1 point] What is the dimension of the end-effector Jacobian, now considering $\begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$ end-effector positions in $SE(2)$.

b. [1 point] Is this system underconstrained, overconstrained, or neither? Why?

c. [1 point] Write out the end-effector Jacobian.

d. [3 points] Assume $\theta_1 = 0.6$, $\theta_2 = 0.6$, and $\theta_3 = 0.4$ (all values in radians). Write out the end-effector velocity $\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}$ for each of:

(i) Joint 1 moving with unit velocity (joints 2 and 3 have zero velocity).

(ii) Joint 2 moving with unit velocity (joints 1 and 3 have zero velocity).

(iii) Joint 3 moving with unit velocity (joints 1 and 2 have zero velocity).

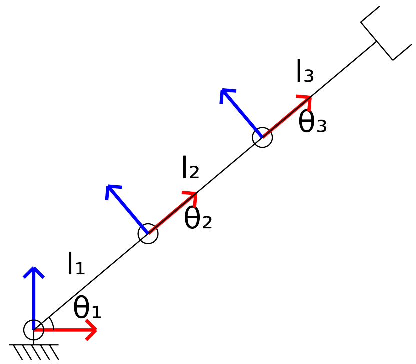
e. [2 points] What is the dimension of the space spanned by these vectors? Are the 1st and 2nd vectors linearly independent? The 1st and 3rd? The 2nd and 3rd? All three?

f. [1 point] What is the rank of J for this configuration?

g. [2 points] When identifying the inverse differential kinematics for this problem, would you use the inverse, the left psuedoinverse, or the right psuedoinverse?

5) RRR Singular Configuration

Consider the following robot



a. [1 point] What is the dimension and rank of the end-effector Jacobian? Consider end effector positions $\begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$. Do the dimension and rank differ? why?

b. [6 points] On separate plots, draw the velocity vector for the end effector caused by

(i) Joint 1 moving with unit velocity (joints 2 and 3 have zero velocity).

(ii) Joint 2 moving with unit velocity (joints 1 and 3 have zero velocity).

(iii) Joint 3 moving with unit velocity (joints 1 and 2 have zero velocity).

The direction of these vectors should be correct, and the relative magnitudes should be approximately correct, but do not worry about exact magnitudes. You can complete this problem by inspection or by using properties of the Jacobian.

c. [2 points] What is the dimension of the space spanned by these vectors? Are the 1st and 2nd vectors linearly independent? The 1st and 3rd? The 2nd and 3rd? All three?

d. [1 point] What is the rank of J for this configuration?

6) Local Inverse Kinematics

Consider a general arm with revolute joints. Assume you already have the kinematic map to the end effector, f, and the Jacobian of the end effector, J.

The robot arm begins with joint angles Θ_0, resulting in an end-effector position of $\mathbf{p}_0 = f(\Theta_0)$. The end effector needs to pick up an object at point $\mathbf{p}_1$.

Note: this is a conceptual problem: do not consider joint limits, and assume that all relevant positions are within the workspace of the robot. Furthermore, all answers are in terms of the given variables and known constants.

a. [2 points] What are the initial joint angle velocities $\dot{\Theta}_0$ that start to move the end effector directly towards $\mathbf{p}_1$?

b. [3 points] Assume the end effector has moved at $\dot{\Theta}_0$ for a small amount of time dt. What joint angles velocities $\dot{\Theta}_{dt}$ will move the end effector from this new starting point directly towards $\mathbf{p}_1$?

c. [2 points] Describe (in a couple of sentences) how you would command the robot to move from $\mathbf{p}_0$ to $\mathbf{p}_1$ in approximately 1 second.

Note that this process describes how differential IK can be used to solve IK *numerically*, given a nearby starting point. We will explore this concept in more depth in the next assignment.

5 Code Questions

Ensure Python and the NumPy and Matplotlib libraries are correctly installed.
For Python installation, visit:

https://www.python.org/downloads/

For NumPy and Matplotlib, install via pip:

```
pip install numpy matplotlib
```

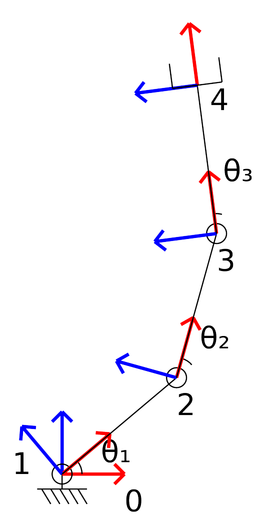
After setting up, download the code folder from the GitHub repository and navigate to that location. Whenever you work on the assignment, run your scripts from inside this directory.

1) [50 points] General Robot Class
Open directory code. For this exercise, we will walk you through the creation of a general Python class representing a fixed-base kinematic chain. You will implement forward kinematics for any RR arm. First, open the file `Robot.py`, and study this file. This is a basic Python class. The idea here is that you can pass parameters to the *constructor* of the robot class, and the returned object then has methods that can be called, and can operate on the values you passed in. For example, the following code:

```
robot = Robot(np.array([[0.55], [0.4]]),\
np.array([[1], [1]]), np.array([[1], [1]]), 0)


frames = robot.fk(np.array([[0], [np.pi/2]]))
```

can be used to find the forward kinematics of a robot with link lengths of 0.55 and 0.4 for $\theta = \begin{bmatrix} 0 \\ \frac{\pi}{2} \end{bmatrix}$



Forward Kinematics Your job is to write forward kinematics that can work for any number of revolute joints in a chain. You will fill in the `forward_kinematics` method in the `Robot` class. This computes H_i^0 for each frame ($i > 0$) in the figure above, including the end effector frame. The code describes the conventions used for the resulting variable. One suggestion that should help is to break this problem into three components:

- First consider frame 1.
- Use a for loop to iterate from frame 2 to $n-1$; use H_{i-1}^0 to help compute frame H_i^0.
- Compute the end effector frame, using H_{n-1}^0 to help.

Verification and Submission To verify your results:

1. Run the `sample_path` function. This will generate a plot and save two CSV files: `calculated_path.csv` and `ground_truth_path.csv`. This is for your own reference, to check whether your output matches the sample test case.
2. Run the local self-check before submitting: from the `local_autograder` directory, run

   ```
   python local_autograder.py
   ```

   This runs your `Robot.fk` over 25 published test arms (chain lengths 2 through 6) and prints PASS/FAIL for each. A FAIL message tells you what to look for:

   - <n>/<total> poses off by > 1 cm — `fk` runs but returns the wrong position; check your homogeneous transforms.
   - raised <ExceptionType>: <message> — `fk` crashed; the message says where.
   - `end-effector path has the wrong shape or is not finite` — your output isn't the expected $(N, 2)$ array, or it contains NaN/inf.
3. The autograder on Gradescope will check your implementation against the same 25 arms and joint angles, but perturbs each one a little and checks against the reference solution instead of a file. If the local check above passes, you should pass on Gradescope too.
4. To submit your work:
   - Run the `create_submission.py` script provided in the `code` directory. This will zip up all necessary files in the `code` folder.
   - Submit the resulting zip file to Gradescope.

Save your sample path plot as a PNG and include it in your writeup. Also include an explanation of your forward kinematics implementation and any challenges you encountered.

6 Submission Checklist

- ☐ Create a PDF of your answers to the written questions with the name `writeup.pdf`.
- ☐ Make sure `writeup.pdf` includes your saved sample-path plot and FK write-up from the Code Questions section.
- ☐ Run `create_submission.py` inside the `code` directory to generate the submission zip file.
- ☐ Upload the resulting `<andrew_id>_hw3.zip` to Gradescope.
- ☐ Upload `writeup.pdf` to Gradescope.